



Trabajo Práctico Integrador

Food Store | Sistema de Gestión de Pedidos de Comida

Alumnos

Juan Franco Cavallieri

Ivan Alejandro Villafañe

Federico Agustin Elias

Materia: Programación II

Comision: 3

22 de Junio, 2026

Índice General de Contenidos

1. Marco Teórico.....	3
1.1 El Lenguaje de Programación Java.....	3
1.2 Fundamentos de Bases de Datos Relacionales y Modelado.....	3
1.3 Conceptos Clave Aplicados (POO y Patrones).....	4
2. Decisiones Técnicas y Arquitectura.....	5
2.1 Estructura y Organización de Paquetes.....	5
2.2 Simulación de Persistencia (Patron Singleton).....	6
2.3 Capturas de Pantalla del Sistema en Ejecución.....	6
3. Dificultades Técnicas y Soluciones Aplicadas.....	8
4. Bibliografía.....	9
5. Enlaces del proyecto.....	9

1. Marco Teórico

El diseño e implementación del sistema "Food Store" requiere asentar conceptualmente los componentes tecnológicos e ingenieriles elegidos para dar respuesta a los requerimientos funcionales del negocio. En el ámbito académico del desarrollo de software, comprender los fundamentos del lenguaje base, las estructuras de organización de datos y las abstracciones del diseño estructurado resulta imperativo para construir una solución extensible, escalable y mantenible en el tiempo.

1.1 El Lenguaje de Programación Java

Como núcleo del backend del sistema, se seleccionó el lenguaje de programación Java. Java es un lenguaje de alto nivel, fuertemente tipado y de propósito general, caracterizado por su enfoque puramente orientado a objetos y su portabilidad multiplataforma gracias a la Máquina Virtual de Java (JVM).

El tipado fuerte garantiza la detección de inconsistencias y errores semánticos durante la fase de compilación, minimizando fallos críticos en tiempo de ejecución (runtime). Asimismo, el ecosistema nativo ofrece un manejo robusto de colecciones y flujos estructurados, permitiendo manipular conjuntos dinámicos de objetos de dominio con un consumo eficiente de recursos y seguridad de memoria gestionada (Garbage Collector).

1.2 Fundamentos de Bases de Datos Relacionales y Modelado

El diseño del almacenamiento de entidades de negocio adopta los principios teóricos del modelo relacional. Las bases de datos relacionales estructuran la información en entidades abstractas representadas matemáticamente como relaciones (tablas), compuestas por tuplas (filas) y atributos (columnas).

Un diseño relacional robusto se rige por restricciones de integridad fundamentales:

Integridad de Entidad (Claves Primarias / PK): Cada registro en una tabla debe poseer un identificador único no nulo (representado por la

propiedad id heredada de Base) que impida la duplicidad de datos.

Integridad Referencial (Claves Foráneas / FK): Las relaciones entre distintas tablas se establecen mediante punteros lógicos. En este sistema, la vinculación entre un Producto y su respectiva Categoría, o entre un Pedido y su Usuario comprador, emula conceptualmente las restricciones de clave foránea de una base de datos física.

1.3 Conceptos Clave Aplicados (POO y Patrones)

La cohesión y el acoplamiento del código fuente se optimizaron mediante la aplicación rigurosa de patrones y conceptos avanzados de ingeniería de software:

Abstracción y Herencia: La unificación de atributos comunes en la clase abstracta Base evita la redundancia de código (principio DRY - Don't Repeat Yourself) y asegura que todas las entidades sigan un ciclo de vida compartido de auditoría y borrado lógico.

Polimorfismo Operacional: A través del uso de contratos funcionales mediante la interfaz Calculable, el sistema desacopla la ejecución de algoritmos numéricos de la estructura física del objeto, permitiendo un diseño limpio y extensible.

Patrón Creacional Singleton: Utilizado para gobernar el acceso al DataStore compartido, evitando colisiones de memoria en entornos multihilo y garantizando una única fuente de verdad operativa para los servicios de la aplicación.

2. Decisiones Técnicas y Arquitectura

2.1 Estructura y Organización de Paquetes

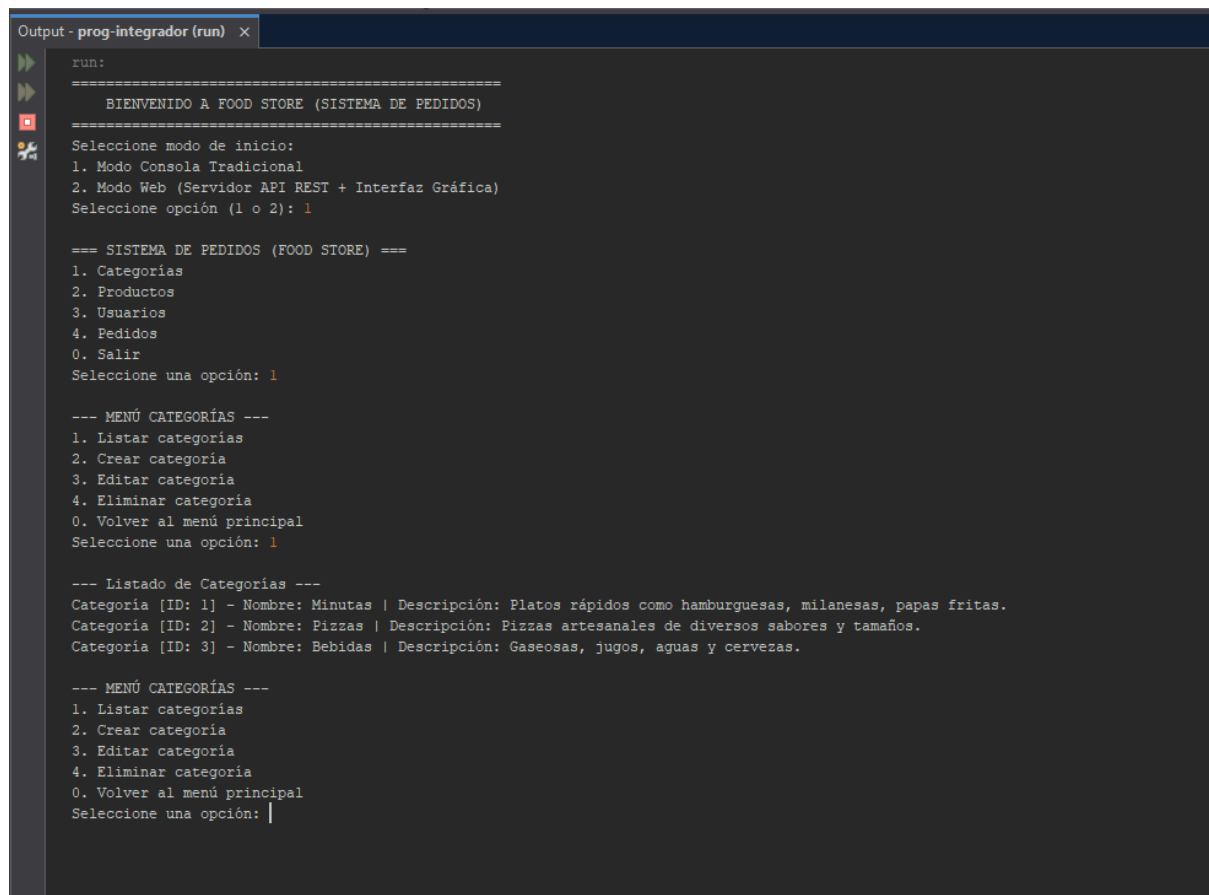
Paquete	Responsabilidad Arquitectónica
integrador.prog2	Punto de entrada a la aplicación (Clase Main). Orquesta el modo de inicio (Consola o Servidor REST).
integrador.prog2.entities	Modelos de datos puros u objetos de dominio que heredan de Base (Usuario, Producto, etc).
integrador.prog2.enums	Enumeradores estáticos que restringen los estados validos del dominio (Estado, FormaPago, Rol).
integrador.prog2.exception	Jerarquía personalizada de manejo estructurado de errores de negocio que extienden de Exception.
integrador.prog2.interfaces	Contratos de abstracción y polimorfismo operacional del sistema (Calculable).
integrador.prog2.service	Capa de lógica de negocio pura. Centraliza las reglas, validaciones de unicidad y mutación de estados.
integrador.prog2.config	Configuración del entorno, simulación de persistencia (DataStore), utilidades (JsonUtil) y red (WebServer).

2.2 Simulación de Persistencia (Patron Singleton)

Ante la ausencia de un motor de bases de datos relacional físico embebido, el almacenamiento de datos en runtime se resolvió mediante la creación de un motor de persistencia simulado en memoria

denominado DataStore. El acceso global a su instancia única se expone mediante un método estático controlado y sincronizado (getInstance()), inicializando categorías, productos y usuarios administradores por defecto al arrancar el sistema.

2.3 Capturas de Pantalla del Sistema en Ejecución



```
Output - prog-integrador (run) x
run:
=====
BIENVENIDO A FOOD STORE (SISTEMA DE PEDIDOS)
=====
Seleccione modo de inicio:
1. Modo Consola Tradicional
2. Modo Web (Servidor API REST + Interfaz Gráfica)
Seleccione opción (1 o 2): 1

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 1

--- MENÚ CATEGORÍAS ---
1. Listar categorías
2. Crear categoría
3. Editar categoría
4. Eliminar categoría
0. Volver al menú principal
Seleccione una opción: 1

--- Listado de Categorías ---
Categoría [ID: 1] - Nombre: Minutas | Descripción: Platos rápidos como hamburguesas, milanesas, papas fritas.
Categoría [ID: 2] - Nombre: Pizzas | Descripción: Pizzas artesanales de diversos sabores y tamaños.
Categoría [ID: 3] - Nombre: Bebidas | Descripción: Gaseosas, jugos, aguas y cervezas.

--- MENÚ CATEGORÍAS ---
1. Listar categorías
2. Crear categoría
3. Editar categoría
4. Eliminar categoría
0. Volver al menú principal
Seleccione una opción: |
```

```

--- MENÚ PRODUCTOS ---
1. Listar todos los productos
2. Listar productos por categoría
3. Crear producto
4. Editar producto
5. Eliminar producto
0. Volver al menú principal
Seleccione una opción: 1

--- Listado de Productos ---
Producto [ID: 1] - Hamburguesa Doble | Precio: $4500,00 | Stock: 20 | Categoría: Minutas | Disponible: true | Desc: Doble carne, doble queso cheddar, lechuga y salsa especial. | Imagen: burger_double.png
Producto [ID: 2] - Papas Fritas Medianas | Precio: $2000,00 | Stock: 50 | Categoría: Minutas | Disponible: true | Desc: Papas fritas bastón crujientes con sal. | Imagen: papas.png
Producto [ID: 3] - Pizza Muzzarella | Precio: $6000,00 | Stock: 15 | Categoría: Pizzas | Disponible: true | Desc: Masa casera, salsa de tomate, muzzarella y aceitunas. | Imagen: pizza_muzza.png
Producto [ID: 4] - Coca Cola 500ml | Precio: $1200,00 | Stock: 100 | Categoría: Bebidas | Disponible: true | Desc: Gaseosa sabor original en botella de plástico. | Imagen: coca.png

--- MENÚ PRODUCTOS ---
1. Listar todos los productos
2. Listar productos por categoría
3. Crear producto
4. Editar producto
5. Eliminar producto
0. Volver al menú principal
Seleccione una opción: |

```

```

--- Editar Producto ---

--- Listado de Productos ---
Producto [ID: 1] - Hamburguesa Doble | Precio: $4500,00 | Stock: 20 | Categoría: Minutas | Disponible: true | Desc: Doble carne, doble queso cheddar, lechuga y salsa especial. | Imagen: burger_double.png
Producto [ID: 2] - Papas Fritas Medianas | Precio: $2000,00 | Stock: 50 | Categoría: Minutas | Disponible: true | Desc: Papas fritas bastón crujientes con sal. | Imagen: papas.png
Producto [ID: 3] - Pizza Muzzarella | Precio: $6000,00 | Stock: 15 | Categoría: Pizzas | Disponible: true | Desc: Masa casera, salsa de tomate, muzzarella y aceitunas. | Imagen: pizza_muzza.png
Producto [ID: 4] - Coca Cola 500ml | Precio: $1200,00 | Stock: 100 | Categoría: Bebidas | Disponible: true | Desc: Gaseosa sabor original en botella de plástico. | Imagen: coca.png
Ingrese el ID del producto a editar: 1
(Deje en blanco/prestione Enter para mantener el valor actual)
Nuevo nombre: Hamburguesa Simple
Nuevo precio: 8000
Nueva descripción: Hamburguesa simple con queso y tomate
Nuevo stock: 45
Nueva imagen: burgasimple.png
Nueva Disponibilidad (1: Disponible, 2: No disponible, Enter para no cambiar):
Selección: 1
¿Desea cambiar la categoría? (Ingrese ID de categoría, o Enter para no cambiar)

--- Listado de Categorías ---
Categoría [ID: 1] - Nombre: Minutas | Descripción: Platos rápidos como hamburguesas, milanesas, papas fritas.
Categoría [ID: 2] - Nombre: Pizzas | Descripción: Pizzas artesanales de diversos sabores y tamaños.
Categoría [ID: 3] - Nombre: Bebidas | Descripción: Gaseosas, jugos, aguas y cervezas.
ID Categoría: 1
✓ Producto actualizado: Producto [ID: 1] - Hamburguesa Simple | Precio: $8000,00 | Stock: 45 | Categoría: Minutas | Disponible: true | Desc: Hamburguesa simple con queso y tomate | Imagen: burgasimple.png

--- MENÚ PRODUCTOS ---
1. Listar todos los productos
2. Listar productos por categoría
3. Crear producto
4. Editar producto
5. Eliminar producto
0. Volver al menú principal
Seleccione una opción: |

```

prog-integrador (run)

```

--- Eliminar Producto ---

--- Listado de Productos ---
Producto [ID: 1] - Hamburguesa Simple | Precio: $8000,00 | Stock: 45 | Categoría: Minutas | Disponible: true | Desc: Hamburguesa simple con queso y tomate | Imagen: burgasimple.png
Producto [ID: 2] - Papas Fritas Medianas | Precio: $2000,00 | Stock: 50 | Categoría: Minutas | Disponible: true | Desc: Papas fritas bastón crujientes con sal. | Imagen: papas.png
Producto [ID: 3] - Pizza Muzzarella | Precio: $6000,00 | Stock: 15 | Categoría: Pizzas | Disponible: true | Desc: Masa casera, salsa de tomate, muzzarella y aceitunas. | Imagen: pizza_muzza.png
Producto [ID: 4] - Coca Cola 500ml | Precio: $1200,00 | Stock: 100 | Categoría: Bebidas | Disponible: true | Desc: Gaseosa sabor original en botella de plástico. | Imagen: coca.png
Ingrese el ID del producto a eliminar: 4
¿Está seguro de eliminar este producto? (S/N): s
✓ Producto eliminado exitosamente (baja lógica).

--- MENÚ PRODUCTOS ---
1. Listar todos los productos
2. Listar productos por categoría
3. Crear producto
4. Editar producto
5. Eliminar producto
0. Volver al menú principal
Seleccione una opción: |

```

```

--- MENÚ USUARIOS ---
1. Listar usuarios
2. Crear usuario
3. Editar usuario
4. Eliminar usuario
0. Volver al menú principal
Seleccione una opción: 3

--- Editar Usuario ---

--- Listado de Usuarios ---
Usuario [ID: 1] - Nombre: Ivan Alejandro | Email: admin@foodstore.com | Celular: 123456789 | Rol: ADMIN
Usuario [ID: 2] - Nombre: Maria Gomez | Email: maria@mail.com | Celular: 987654321 | Rol: USUARIO
Usuario [ID: 3] - Nombre: Juan Perez | Email: juan@mail.com | Celular: 555123456 | Rol: USUARIO
Ingrese el ID del usuario a editar: 3
(Deje en blanco/presione Enter para mantener el valor actual)
Nuevo nombre: Martin
Nuevo apellido: Perez
Nuevo email: mperez@mail.com
Nuevo celular: 2234556678
Nueva contraseña: 1234
Nuevo Rol (1: ADMIN, 2: USUARIO, Enter para no cambiar):
Seleccione: 2
✓ Usuario actualizado: Usuario [ID: 3] - Nombre: Martin Perez | Email: mperez@mail.com | Celular: 2234556678 | Rol: USUARIO

--- MENÚ USUARIOS ---
1. Listar usuarios
2. Crear usuario
3. Editar usuario
4. Eliminar usuario
0. Volver al menú principal
Seleccione una opción:

```

3. Dificultades Técnicas y Soluciones Aplicadas

Gestión del Ciclo de Vida para Evitar Corrupción de Memoria (Baja Lógica)

Dificultad: Al tener todas las entidades relacionadas en listas de memoria cruzadas, eliminar físicamente un objeto (por ejemplo, haciendo `dataStore.getCategorias().remove(categoria)`) resultaba muy peligroso. Si existían productos que seguían apuntando a ese objeto eliminado, se generarían inconsistencias graves y errores de referencia nula (`NullPointerException`) al momento de listarlos o procesarlos.

Solución: Se prohibió la eliminación física en las listas del `DataStore`. En su lugar, se implementó un mecanismo de ****Baja Lógica**** utilizando el atributo booleano `isEliminado` centralizado en la clase `Base`. Además, se sumaron validaciones estructurales: al intentar dar de baja una Categoría, el servicio verifica primero la lista de productos; si encuentra alguno activo vinculado a dicha categoría, bloquea la operación lanzando una excepción de negocio, garantizando la estabilidad del árbol de objetos.

Protección del Encapsulamiento en Colecciones (Copias Defensivas)

Dificultad: El DataStore centraliza todas las colecciones maestras (ej. la lista de productos). Si métodos como listarProductos() devolvían directamente la referencia a datastore.getProductos(), se rompía el principio de encapsulamiento: cualquier clase externa podría ejecutar un .add() o .clear() saltándose todas las validaciones de negocio e inyectando datos corruptos.

Solución: Se implementó un patrón de acceso seguro mediante la instanciación de nuevas colecciones. Cada vez que la capa de servicio solicita un listado, el método instancia un nuevo ArrayList<Producto>, itera sobre el repositorio principal, filtra únicamente las entidades válidas (sin baja lógica) y retorna esa nueva estructura generada dinámicamente. De este modo, la memoria original del DataStore queda blindada contra manipulaciones indebidas por referencia cruzada.

4. Bibliografía

IBM Docs. (s.f.). *Guía de referencia de Java: Conceptos de programación orientada a objetos y estructuras de datos.* Recuperado de: <https://www.ibm.com/docs/es/java>

Microsoft Learn. (s.f.). *Documentación de Java para desarrolladores: Arquitectura, gestión de memoria y buenas prácticas.* Recuperado de: <https://learn.microsoft.com/es-es/java/>

5. Enlaces del proyecto

Link al repositorio: <https://github.com/IvanVillafane/integrador-prog2>

Link al video explicativo: <https://youtu.be/PmcZY5AAyos>